

Practical 6 OS

```
nishanth@LAPTOP-6Q3ON07H: ~/practical6
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/request_fifo"
#define TEXT_SIZE 256

typedef struct {
    pid_t pid;
    char message[TEXT_SIZE];
} Request;

int main(void)
{
    Request request;

    if (mkfifo(SERVER_FIFO, 0666) == -1 && errno != EEXIST) {
        perror("mkfifo");
        return EXIT_FAILURE;
    }

    printf("Server started. Waiting for clients...\n");

    int server_fd = open(SERVER_FIFO, O_RDONLY);
    if (server_fd == -1) {
        perror("open");
        unlink(SERVER_FIFO);
        return EXIT_FAILURE;
    }

    while (1) {
        ssize_t bytes = read(server_fd, &request, sizeof(request));
        if (bytes == -1) {
            if (errno == EINTR) {
                continue;
            }
            perror("read");
            break;
        }
    }
}
```

```
        perror("read");
        break;
    }

    if (bytes != sizeof(request)) {
        continue;
    }

    printf("Client %d sent: %s\n",
           (int)request.pid, request.message);

    /* Process the message by converting it to uppercase. */
    for (size_t i = 0; request.message[i] != '\0'; i++) {
        request.message[i] =
            (char)toupper((unsigned char)request.message[i]);
    }

    char client_fifo[128];

    snprintf(client_fifo, sizeof(client_fifo),
             "/tmp/response_%d", (int)request.pid);

    int client_fd = open(client_fifo, O_WRONLY);

    if (client_fd == -1) {
        perror("open client FIFO");
        continue;
    }

    if (write(client_fd, request.message,
              strlen(request.message) + 1) == -1) {
        perror("write");
    }

    close(client_fd);
}

close(server_fd);
unlink(SERVER_FIFO);

return EXIT_SUCCESS;
}
```

```
nishanth@LAPTOP-6Q3ON07H:~/practical6$ ./program1 &
[1] 611
Server started. Waiting for clients...
nishanth@LAPTOP-6Q3ON07H:~/practical6$ ./program1 &
[2] 626
Server started. Waiting for clients...
nishanth@LAPTOP-6Q3ON07H:~/practical6$ nano program2.c
nishanth@LAPTOP-6Q3ON07H:~/practical6$ gcc -Wall -Wextra program2.c -o program2
/usr/bin/x86_64-linux-gnu-ld.bfd: /usr/lib/gcc/x86_64-linux-gnu/15/../../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
nishanth@LAPTOP-6Q3ON07H:~/practical6$ nano program2.c
nishanth@LAPTOP-6Q3ON07H:~/practical6$
nishanth@LAPTOP-6Q3ON07H:~/practical6$ gcc -Wall -Wextra program2.c -o program2
nishanth@LAPTOP-6Q3ON07H:~/practical6$ ./program1 &
[3] 697
Server started. Waiting for clients...
nishanth@LAPTOP-6Q3ON07H:~/practical6$ ./program2 "hello from client"
Client 707 sent: hello from client
Server response: HELLO FROM CLIENT
nishanth@LAPTOP-6Q3ON07H:~/practical6$
```

```

#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

static volatile sig_atomic_t sigint_received = 0;
static volatile sig_atomic_t sigusr1_received = 0;
static volatile sig_atomic_t sigterm_received = 0;

static void signal_handler(int signal_number)
{
    if (signal_number == SIGINT) {
        sigint_received = 1;
    } else if (signal_number == SIGUSR1) {
        sigusr1_received = 1;
    } else if (signal_number == SIGTERM) {
        sigterm_received = 1;
    }
}

int main(void)
{
    struct sigaction action = {0};

    action.sa_handler = signal_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGINT, &action, NULL) == -1) {
        perror("sigaction SIGINT");
        return EXIT_FAILURE;
    }

    if (sigaction(SIGUSR1, &action, NULL) == -1) {
        perror("sigaction SIGUSR1");
        return EXIT_FAILURE;
    }

    if (sigaction(SIGTERM, &action, NULL) == -1) {
        perror("sigaction SIGTERM");
        return EXIT_FAILURE;
    }

    printf("Signal handling program started\n");
}

```

```

printf("PID: %d\n", (int)getpid());
printf("Press Ctrl+C for SIGINT\n");
printf("Use: kill -USR1 %d\n", (int)getpid());
printf("Use: kill -TERM %d\n", (int)getpid());

while (!sigterm_received) {
    pause();

    if (sigint_received) {
        sigint_received = 0;
        printf("SIGINT received and processed\n");
    }

    if (sigusr1_received) {
        sigusr1_received = 0;
        printf("SIGUSR1 received and processed\n");
    }
}

printf("SIGTERM received. Exiting cleanly\n");

return EXIT_SUCCESS;
}

```

```

nishanth@LAPTOP-6Q30N07H:~/practical6$ nano program3.c
nishanth@LAPTOP-6Q30N07H:~/practical6$ gcc -Wall -Wextra program3.c -o program3
nishanth@LAPTOP-6Q30N07H:~/practical6$ ./program3 &
[4] 747
Signal handling program started
PID: 747
Press Ctrl+C for SIGINT
Use: kill -USR1 747
Use: kill -TERM 747
nishanth@LAPTOP-6Q30N07H:~/practical6$ kill -USR1 747
SIGUSR1 received and processed
nishanth@LAPTOP-6Q30N07H:~/practical6$ kill -INT 747
SIGINT received and processed
nishanth@LAPTOP-6Q30N07H:~/practical6$ kill -TERM 747
SIGTERM received. Exiting cleanly
[4]+  Done                  ./program3
nishanth@LAPTOP-6Q30N07H:~/practical6$

```